

Programmation synchrone

T. Carle
2022-2027



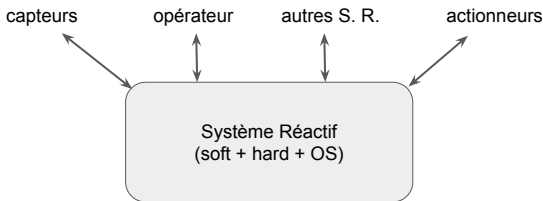
Remerciements

Merci à Jean-Baptiste Raclet et Pascal Raymond pour m'avoir permis de réutiliser largement leurs supports de cours.

Systèmes réactifs

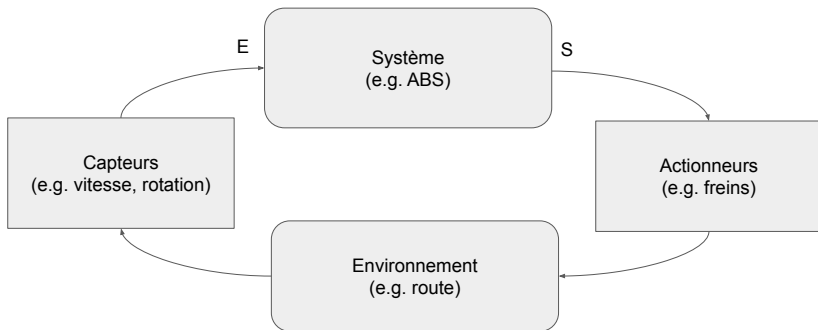
- On s'intéresse à des systèmes **réactifs** : ils réagissent en permanence à leur **environnement**, avec des contraintes de temps.
- Ex : systèmes de contrôle/commande industriels, transports, téléphones

Systemes réactifs



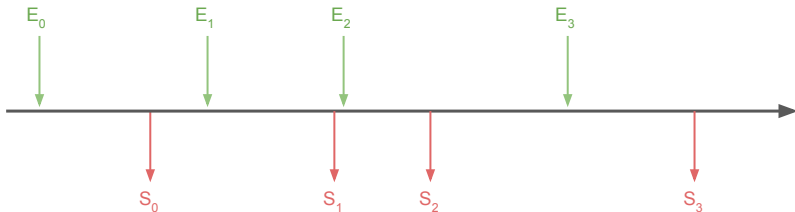
- Beaucoup d'interactions = beaucoup de problèmes
- => Focus sur les propriétés fonctionnelles, en particulier sur les **communications** (entre fonctions/tâches)
- Objectif des langages synchrones : permettre de **spécifier** de façon **non ambiguë** le **schéma de communications** entre les différentes activations (instances) des différents sous-systèmes
 - A charge pour le compilateur de générer le code imposant ce schéma

Systemes réactifs



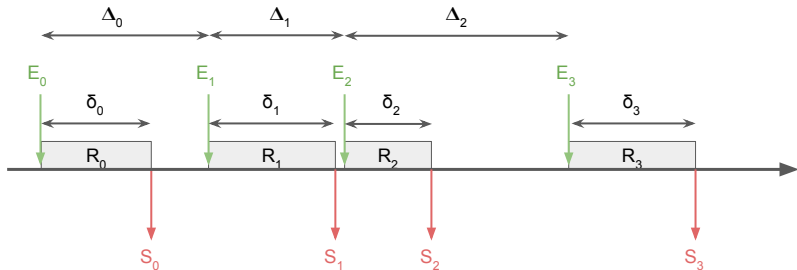
- Exécution du système = séquence "infinie" de réactions

Systemes réactifs



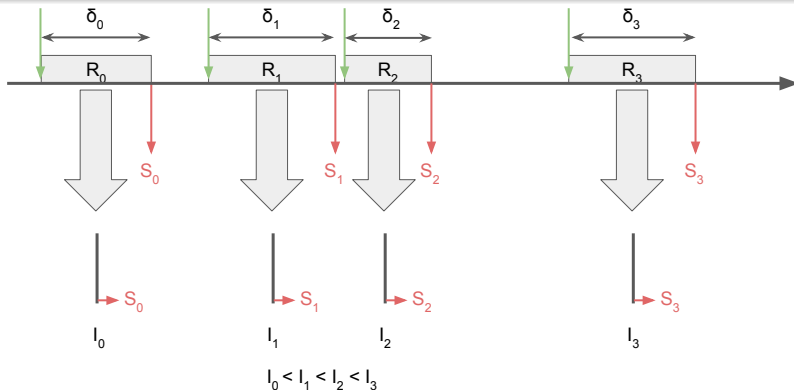
- Comportement idéal : séquence de réactions (E/S alternées)
- **Déterminisme fonctionnel** :
 - Une séquence d'entrée donne toujours la même séquence de sortie
 - $S_i = \phi(E_0, \dots, E_i)$

Hypothèse synchrone



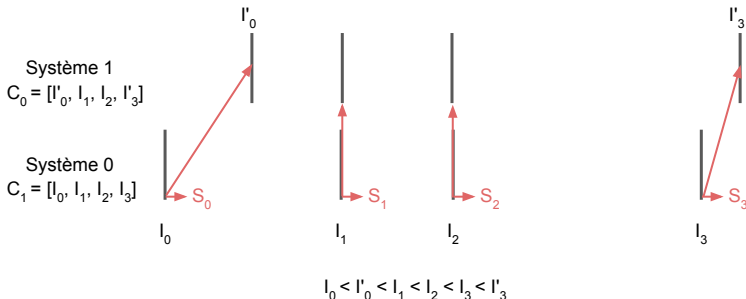
- Hypothèse synchrone :
 - Les communications sont **instantannées et non bloquantes**
 - Le calcul de la réaction est **instantané**
- Notion de "temps" logique i.e. séquence de réactions déclenchées par l'arrivée des entrées
 - Instants vs dates : notion d'"**horloge logique**"
 - Comme dans un circuit, on met à jour les entrées **uniquement** à chaque coup d'horloge
- L'hypothèse est vérifiée si : $\max(\delta_i) < \min(\Delta_j)$ (cond. suffisante)

Hypothèse synchrone



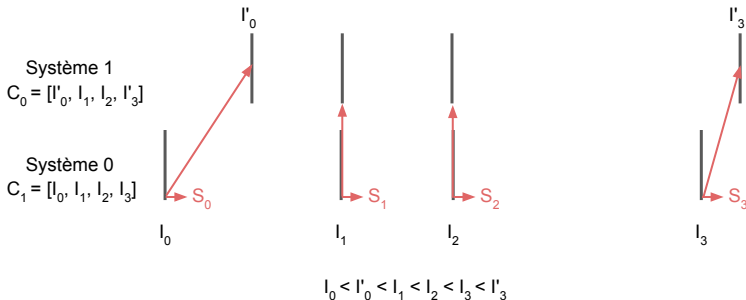
- Hypothèse synchrone :
 - Le calcul de la réaction est instantané
- Notion de "temps" logique i.e. séquence de réactions déclenchées par l'arrivée des entrées
 - Temps discret vs temps dense (instants/précédences vs dates/distances)

Concurrence logique



- Hypothèse synchrone :
 - Les communications sont instantannées et non bloquantes
- Notion de "temps" logique i.e. séquence de réactions déclenchées par l'arrivée des entrées
 - Instants vs dates : notion d'"**horloge logique**"
 - Comme dans un circuit, on met à jour les entrées **uniquement** à chaque coup d'horloge

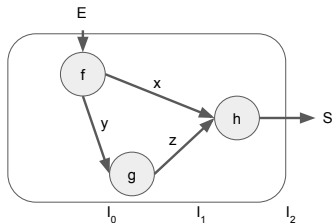
Concurrence logique



- Schéma de communication déterministe dès la spécification du système
 - Facilite les campagnes de tests
 - Permet de la vérification formelle (raisonnement sur les instants)
 - Permet de calculer des latences de bout en bout
 - Facilite la certification des systèmes critiques

Concurrence logique

Exercice : On considère le système suivant :



$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$

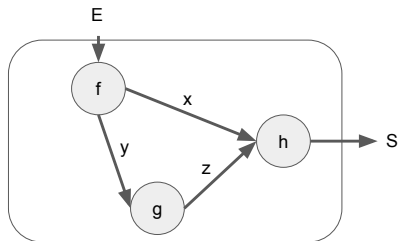
f

g

h

- ① Déterminer le schéma de communications entre les 3 sous-systèmes
- ② Déterminer les équations $S_i = h(x_j, z_k)$ pour chaque sortie

Concurrence logique



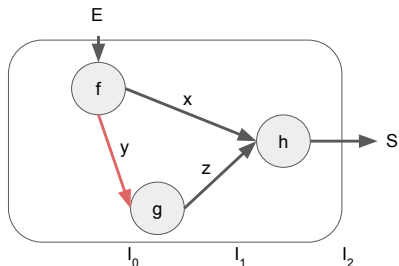
$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$

	l_0	l_1	l_2	l_3	l_4	l_5	l_6	l_7
f								
g								
h								

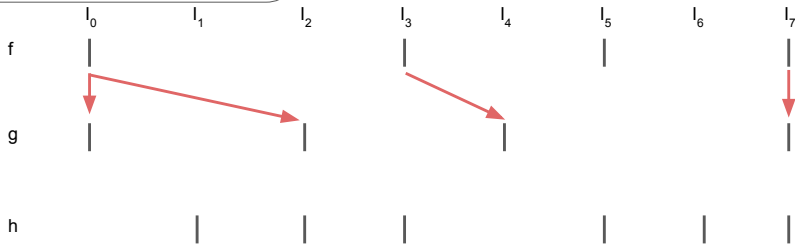
Concurrence logique



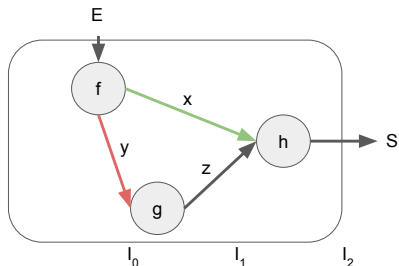
$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$



Concurrence logique



$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$

f

g

h

l_0

l_1

l_2

l_3

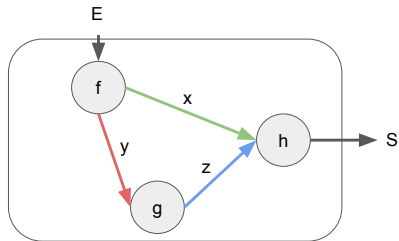
l_4

l_5

l_6

l_7

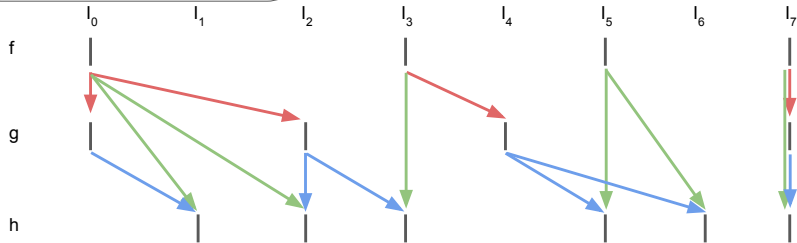
Concurrence logique



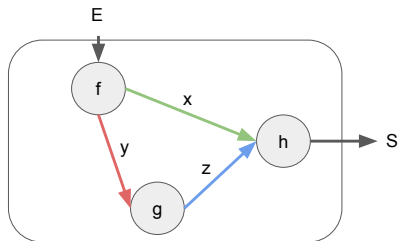
$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$



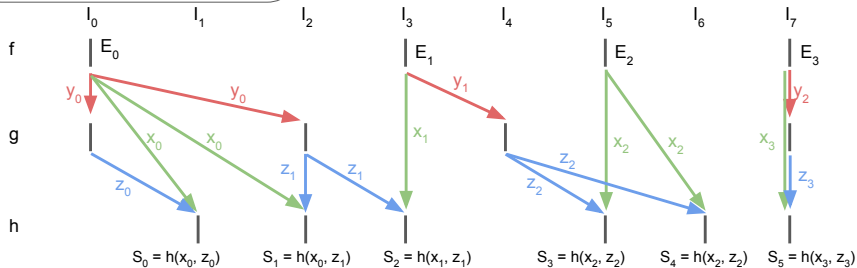
Concurrence logique



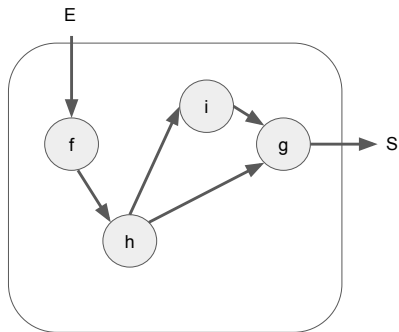
$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$

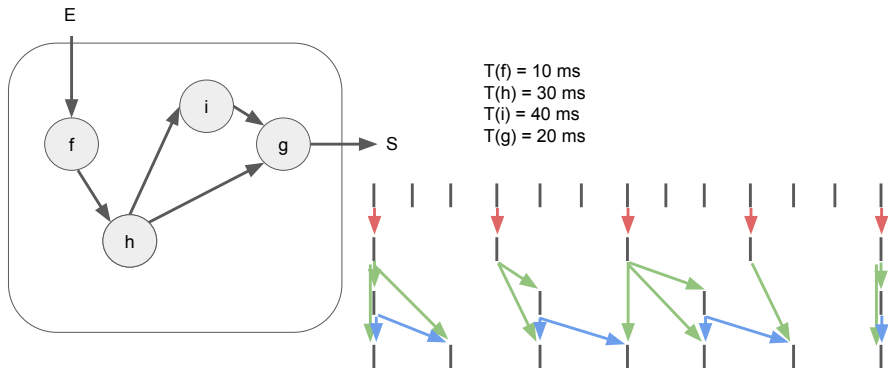


Concurrence logique



$T(f) = 10 \text{ ms}$
 $T(h) = 30 \text{ ms}$
 $T(i) = 40 \text{ ms}$
 $T(g) = 20 \text{ ms}$

Concurrence logique



Concurrence logique

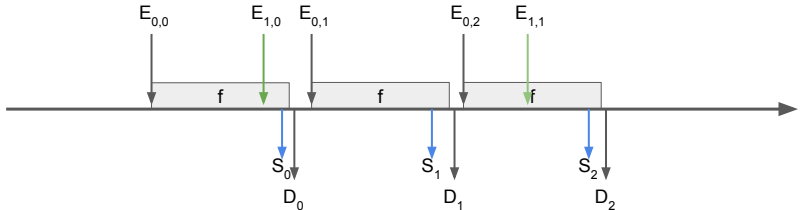


Retour à la réalité

- Systèmes réels plus complexes que l'exemple :
 - Entrées multiples
 - Variabilité temporelle
 - Multi-systèmes
 - Systèmes co-dépendants
- Compilation de la fonction de réaction

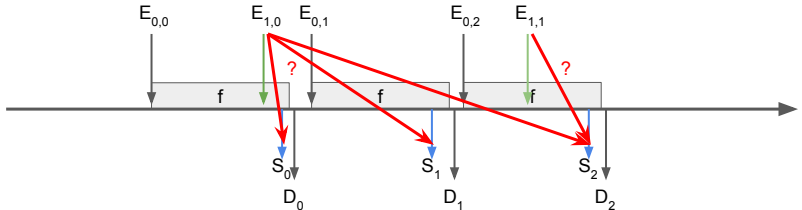
Problèmes

```
void f(){ //E0 et E1 variables globales
...;
int x = E0;
...;
if(E1) {
    x = 3*x+2;
}
else {
    ...;
    x = x*x;
};
write(S0, x);
... }
```



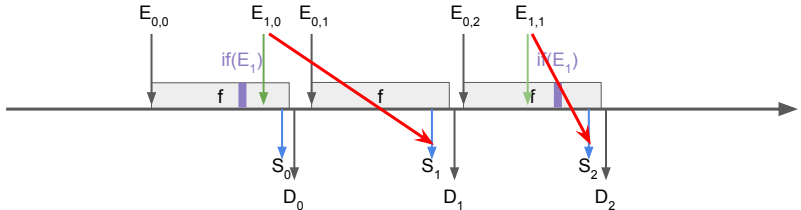
Problèmes

```
void f(){ //E0 et E1 variables globales
...;
int x = E0;
...;
if(E1) {
    x = 3*x+2;
}
else {
    ...;
    x = x*x;
};
write(S0, x);
... }
```



Problèmes

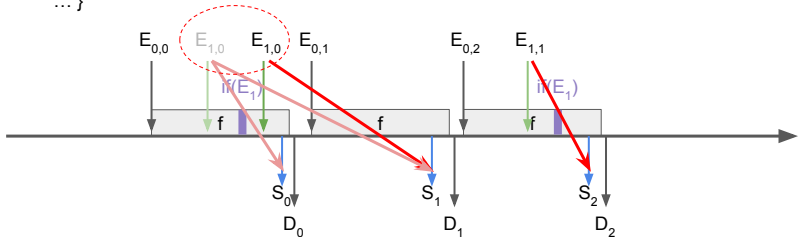
```
void f(){ //E0 et E1 variables globales
...;
int x = E0;
...;
if(E1) {
    x = 3*x+2;
}
else {
    ...;
    x = x*x;
};
write(S0, x);
... }
```



Problèmes

```
void f(){ //E0 et E1 variables globales
...;
int x = E0;
...;
if(E1) {
    x = 3*x+2;
}
else {
    ...;
    x = x*x;
};
write(S0, x);
... }
```

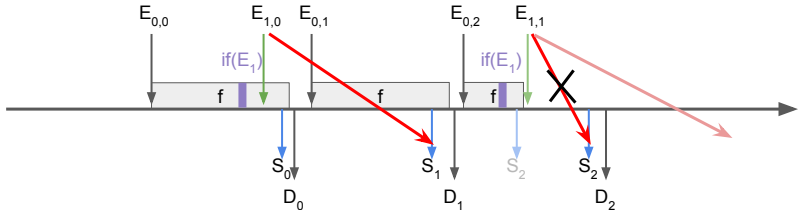
Problème 1 : dépendance à la date des entrées



Problèmes

```
void f(){ //E0 et E1 variables globales
...;
int x = E0;
...; // Variabilité
if(E1) {
    x = 3*x+2;
}
else {
    ...;
    x = x*x;
};
write(S0, x);
... }
```

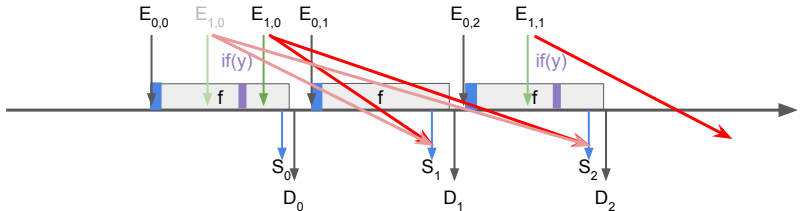
Problème 2 : dépendance à la variabilité temporelle



Problèmes

```
void f(){ //E0 et E1 variables globales
  int x = E0;
  int y = E1;
  ...;
  if(y) {
    x = 3*x+2;
  }
  else {
    ...;
    x = x*x;
  };
  write(S0, x);
  ... }
```

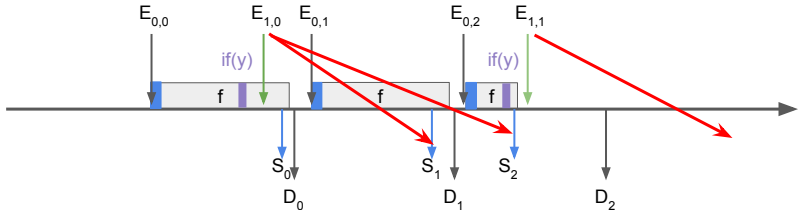
Problème 1 : dépendance à la date des entrées
=> Régulé



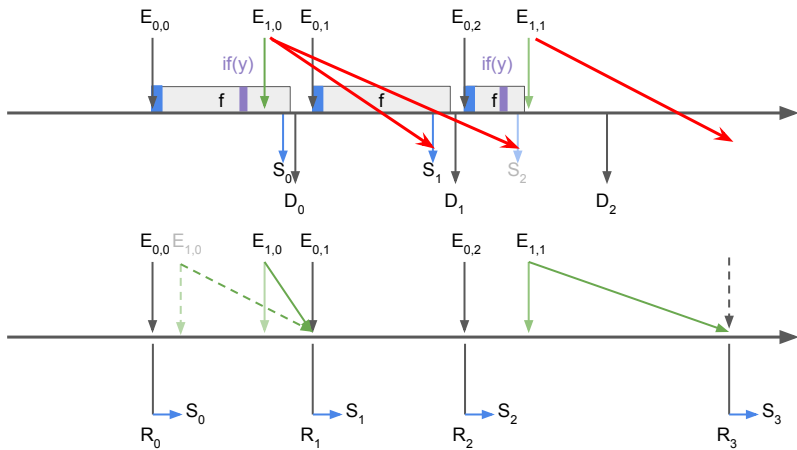
Problèmes

```
void f(){ //E0 et E1 variables globales
  int x = E0;
  int y = E1;
  ...;
  if(y) {
    x = 3*x+2;
  }
  else {
    ...;
    x = x*x;
  };
  write(S0, x);
  ... }
```

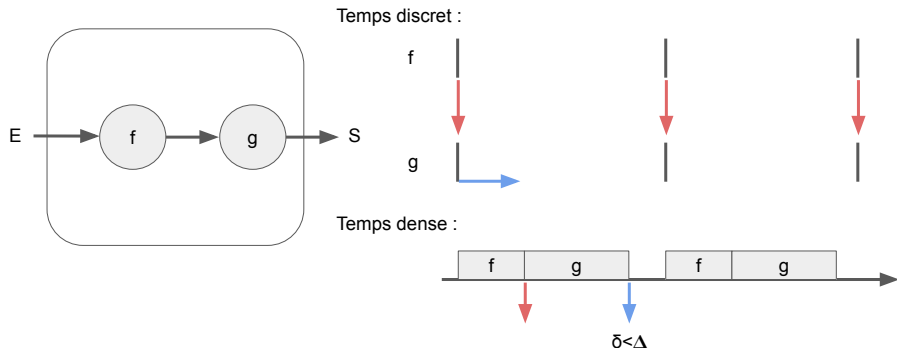
Problème 2 : dépendance à la variabilité temporelle
=> Régulé



Problèmes

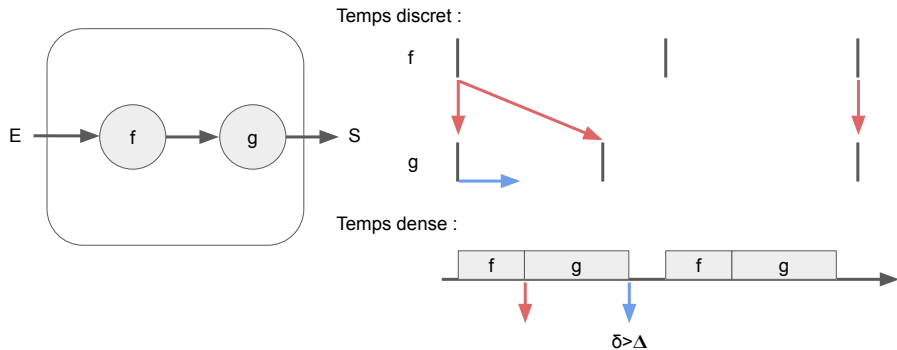


Problèmes



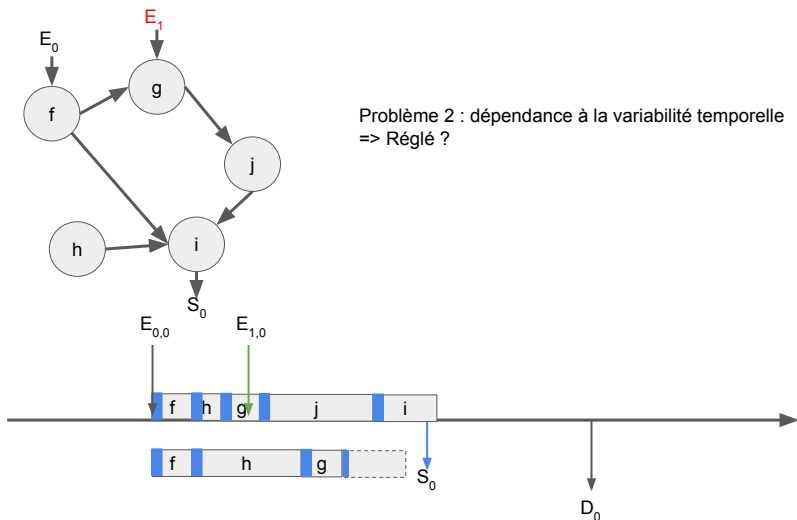
- L'ordonnancement **causal** (suivant les dépendances) permet d'implémenter l'hypothèse des communications instantanées entre tâches du système :
 - Les données sont disponibles dès qu'on en a besoin
 - Ex : du point de vue de *g*, c'est comme si les données étaient disponibles au début de la réaction

Problèmes

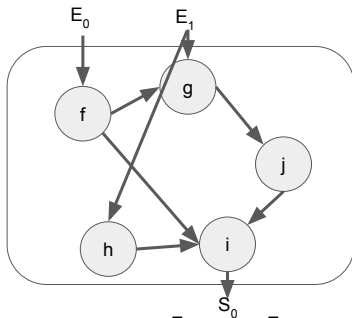


- L'ordonnancement **causal** (suivant les dépendances) permet d'implémenter l'hypothèse des communications instantanées :
 - Lorsque les sous-systèmes ne vont pas à la même vitesse, le Δ de l'hypothèse est calculé sur l'ensemble des instants du système

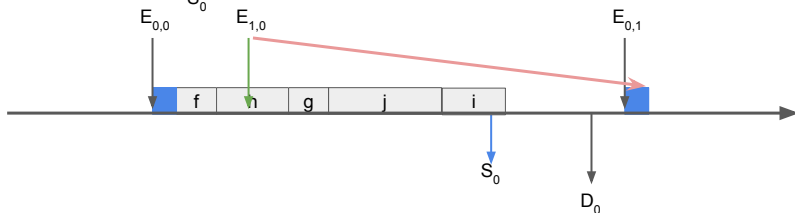
Problèmes



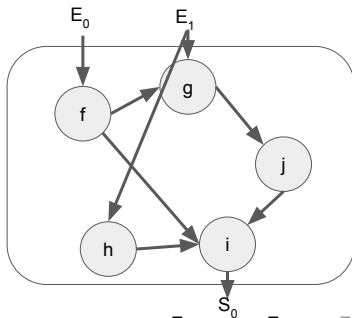
Problèmes



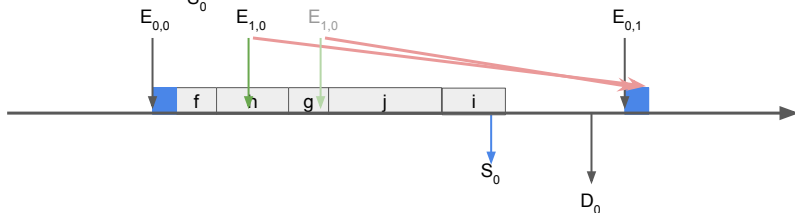
Solution : synchroniser la lecture des entrées au niveau du système



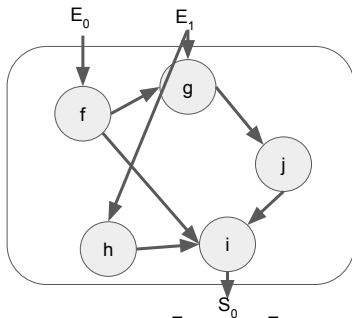
Problèmes



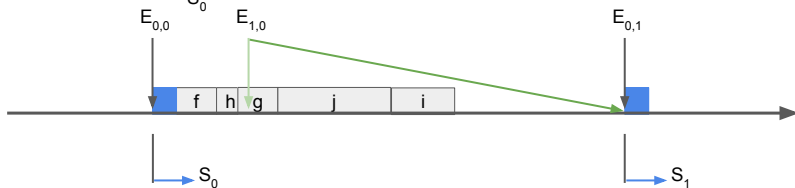
Solution : synchroniser la lecture des entrées au niveau du système



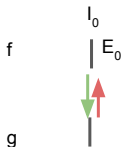
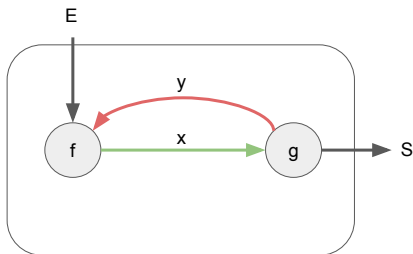
Problèmes



Solution : synchroniser la lecture des entrées au niveau du système

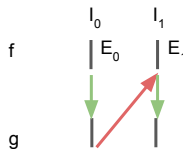
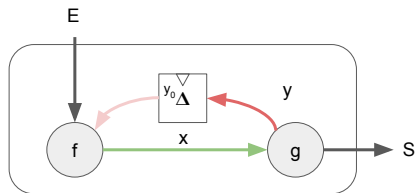


Encore des problèmes



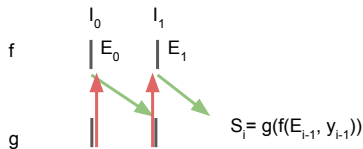
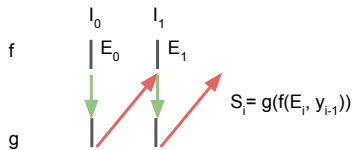
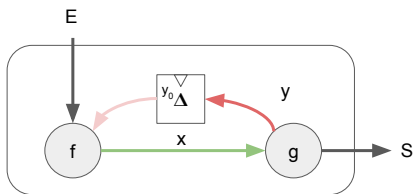
- Hypothèse synchrone :
 - Les communications sont instantannées et non bloquantes
 - Le calcul de la réaction est instantané
- f et g sont mutuellement récursives au sein de la même réaction
- $\Rightarrow$ problème de **causalité**

Encore des problèmes



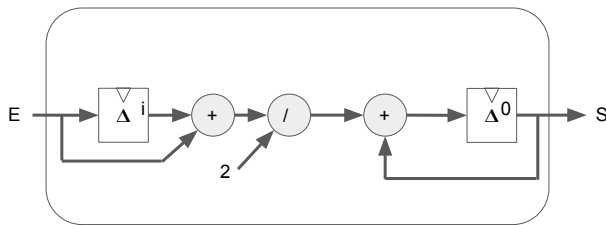
- Hypothèse synchrone :
 - Les communications sont instantannées et non bloquantes
 - Le calcul de la réaction est instantané
- Casser la récursivité dans la spécification avec des **retards synchrones**
 - Analogues à des registres en circuit
 - Le compilateur rejette les programmes non causaux

Encore des problèmes



Retards synchrones

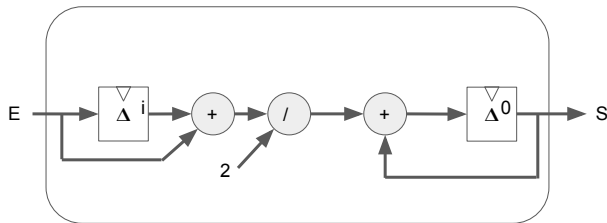
Exercice :



- 1 Définir la fonction f qui relie les sorties successives S_i aux entrées E_i .
- 2 Que calcule cette fonction ?

Retards synchrones

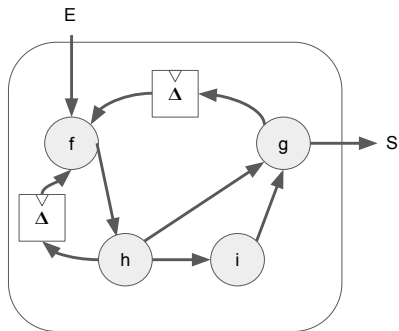
Exercice :



- 1 Définir la fonction f qui relie les sorties successives S_i aux entrées E_i .
 $S_0 = 0$ et $S_i = (E_i + E_{i-1})/2 + S_{i-1}$
- 2 Que calcule cette fonction ?
C'est un intégrateur

Retards synchrones

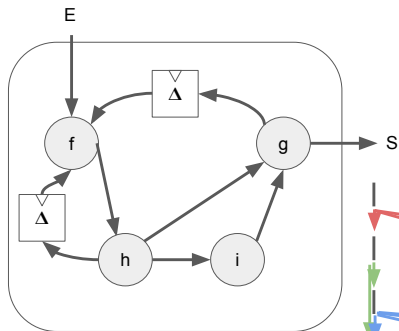
Exercice :



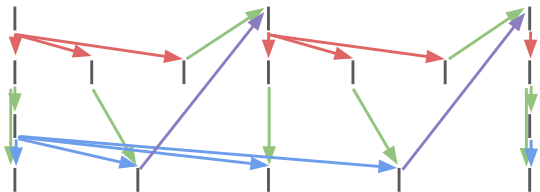
$T(f) = 60 \text{ ms}$
 $T(h) = 20 \text{ ms}$
 $T(i) = 120 \text{ ms}$
 $T(g) = 30 \text{ ms}$

Retards synchrones

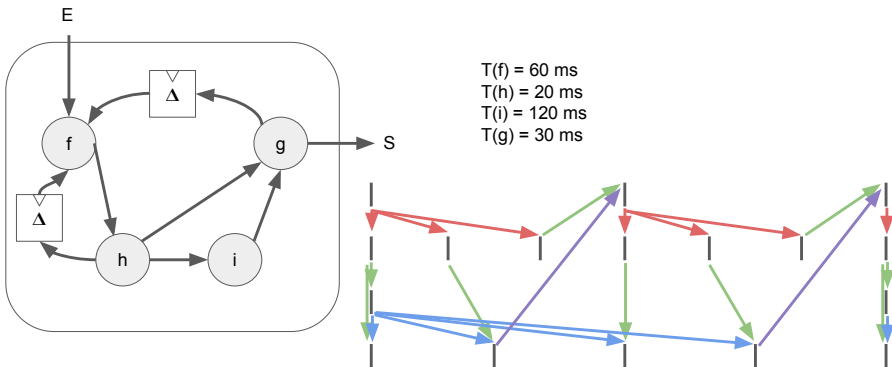
Exercice :



$T(f) = 60 \text{ ms}$
 $T(h) = 20 \text{ ms}$
 $T(i) = 120 \text{ ms}$
 $T(g) = 30 \text{ ms}$

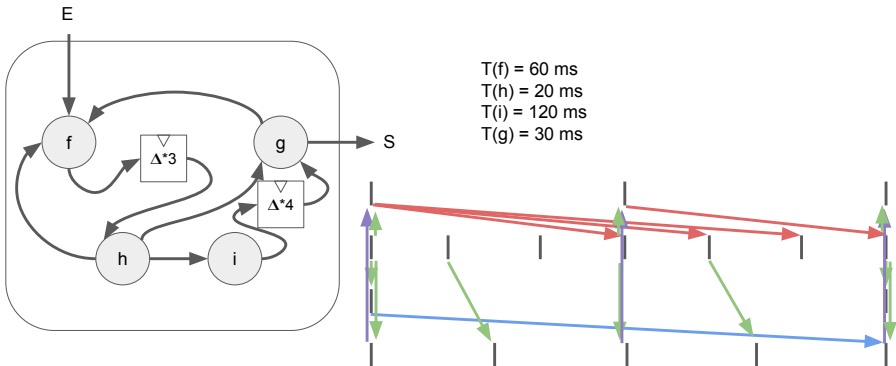


Retards synchrones



- En pratique, une tâche à longue période est souvent plus lente qu'une tâche à période rapide
 - Dans notre exemple f communique sa sortie à h sans retard
 - Cela peut poser problème pour l'hypothèse synchrone $\max(\delta_i) < \min(\Delta_j)$
 - Ici, on doit faire f ; h ; i ; g en moins de 20ms

Retards synchrones



- En pratique, une tâche à longue période est souvent plus lente qu'une tâche à période rapide
 - On va plutôt déployer les retards en sortie des tâches lentes vers les tâches rapides (si c'est possible)
 - Attention aux **rythmes** de production et de consommation des données !
 - En Lustre, géré explicitement par le programmeur

Plus de problème



Fonction de réaction

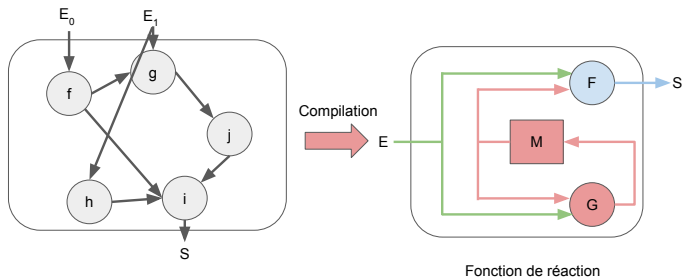
- La forme du code généré influence la validité de l'hypothèse synchrone
- Implementations multi-threads?
 - Problèmes de communications (y compris avec rendez-vous Ada)
 - Problèmes de temps-réel liés à l'ordonnancement (e.g. inversion de priorité)
- Pour simplifier, on vise une solution standard pour les systèmes réactifs
 - => **Fonctions de réaction**
 - Plus statique que le multi-threads
 - Basé sur de l'ordonnancement statique (solution de type compilation) qui respecte la causalité
- Les systèmes distribués ou multicœurs rajoutent de la complexité
 - Communications non-immédiates
 - Synchronisation des calculs pour garantir la causalité

Fonction de réaction

- On voit le système comme une machine de Mealy :
 - Un état interne M (de taille bornée statiquement), qui correspond aux retards synchrones
 - Une séquence d'entrées E
 - Une fonction $G(E, M)$ de mise à jour de l'état interne
 - Une fonction $F(E, M)$ de production des sorties

Fonction de réaction

- On voit le système comme une machine de Mealy :



- Une réaction se décompose comme suit :
 - Lecture des entrées E de la réaction
 - Calcul des sorties de la réaction : $S = F(E, M)$
 - Mise à jour de l'état interne : $M = G(E, M)$
 - Écriture des sorties S de la réaction

Langages synchrones

- Les langages synchrones permettent de spécifier le comportement d'applications embarquées réactives :
 - **Calculs** à réaliser
 - **Horloges** et communications, **rythmes** de production/consommation
- Les compilateurs associés permettent de générer le code correspondant et d'appliquer à différents niveaux des vérifications et des optimisations :
 - **Causalité** et endochronie : une donnée dont j'ai besoin à une réaction est-elle présente et valide ? (vérification niveau spécification)
 - Ordonnancement statique causal : **respecter les dépendances** au sein d'une réaction (les données intermédiaires sont présentes quand on en a besoin)
 - Vérification de la **borne d'utilisation mémoire** (production/consommation)
 - Synchronisations en cas de multi-thread/distribué/multicoeur

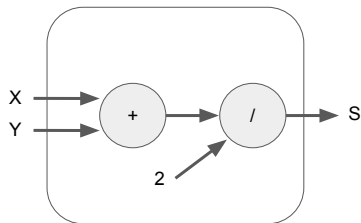
Langages Synchrones

- Les LS partagent les mêmes principes basiques
 - La synchronie (temps discret)
 - La concurrence logique
 - De la compilation vers du code séquentiel simple
- Mais ont différents styles
 - Déclaratif/flot-de-données
 - Textuel (Lustre, Signal) ou Graphique (SCADE, SynDEx)
 - Impératif/séquentiel
 - Textuel (Esterel) ou Graphique (SynchCharts)

Lustre

- Approche **flot-de-données**

- Un programme est un ensemble d'opérateurs reliés par des câbles
- Classique en automatique et en circuit



```
node Average(X, Y : int)
returns (S : int);
let
    S = (X + Y) / 2 ;
tel
```

- Synchrone : temps discret ($\mathbb{N}$)
- $\forall t \in \mathbb{N}, S_t = (x_t + y_t)/2$
- Parallélisme : les noeuds sont **concurrents**

```
node Average(X, Y : int)
returns (S : int);
var Z : int;
let
    S = Z / 2 ;
    Z = X + Y ;
tel
```

- **Déclaratif** : ensemble d'équations (pas une séquence d'assignations)
- Une **seule** équation par sortie et variable intermédiaire
- Les variables sont des séquences infinies de valeurs

- Types de base : **int**, **bool**, **real**
- Constantes :
 - $2 \equiv 2, 2, 2, \dots$
 - $true \equiv true, true, \dots$
- Opérateurs point à point
 - $X \equiv X_0, X_1, X_2, \dots, Y \equiv Y_0, Y_1, Y_2, \dots$
 $X + Y \equiv X_0 + Y_0, X_1 + Y_1, X_2 + Y_2, \dots$
- Les opérateurs classiques sont disponibles

- Opérateur **if-then-else**

```
node Max(X, Y : real) returns (S : real);  
let  
    S = if (A >= B) then A else B;  
tel
```

- Opérateur **fonctionnel** ($\neq$ contrôle) : 1 seule équation par variable

```
node Max(X, Y : real) returns (S : real);  
Let  
    if (A >= B) then S = A ;  
    else S = B;  
tel
```

- Opérateur de **retard** : **pre**

X	X_0	X_1	X_2	X_3	X_4	...
pre X	nil	X_0	X_1	X_2	X_3	...

i.e. $pre(X)_0$ non défini et $\forall i > 0, pre(X)_i = X_{i-1}$

- Initialisation** : $->$

X	X_0	X_1	X_2	X_3	X_4	...
Y	Y_0	Y_1	Y_2	Y_3	Y_4	...
$X -> Y$	X_0	Y_1	Y_2	Y_3	Y_4	...

i.e. $(X -> Y)_0 = X_0$ et $\forall i > 0, (X -> Y)_i = y_i$

- Exercice : noeud Compteur (entrée $t : \text{int}$) et avec sortie $S = 0, 1, 2, 3, \dots$

- Exercice : noeud Compteur (entrée $t : \text{int}$) et avec sortie $S = 0, 1, 2, 3, \dots$ ($S_0 = 0$ et $\forall i > 0, S_i = S_{i-1} + 1$)

```
node Compteur(t: int) returns (S : int);  
let  
    S = 0 -> pre S + 1;  
tel
```

- Exercice : noeud $\text{edge}(x)$: edge renvoie vrai si x vient de générer un front montant (false $\rightarrow$ true)

- Exercice : noeud `edge(x)` : `edge` renvoie vrai si `x` vient de générer un front montant (`false -> true`)

```
node Edge (X : bool) returns (S : bool);  
let  
    S = false -> X and not pre X;  
tel
```

- Exercice : noeud minmax(x) : renvoie le min et le max des x vus jusqu'à présent (2 sorties)

- Exercice : noeud minmax(x) : renvoie le min et le max des x vus jusqu'à présent (2 sorties)

```
node MinMax(X : int)
returns (min : int, max : int);
let
  min = X -> if (X > pre min) then pre min else X;
  max = X -> if (X < pre max) then pre max else X;
tel
```

- Attention aux boucles combinatoires ! (**causalité**)

$s = 0 \rightarrow s + 1;$

$x = 1 / (2 - x);$

- Ces équations ne se résolvent pas par étape (construire la sortie en fonction de l'état courant)

- Détection des boucles :

```
X = if C then Y else A;  
Y = if C then B else X;
```

- Boucle syntaxique mais pas sémantique
- $X = Y = \text{if } C \text{ then } B \text{ else } A$ est la seule solution
- Lustre refuse toutes les boucles **syntaxiques**, même si elles sont des "fausses" boucles

Exercices :

- Ecrivez en lustre l'équation générant la suite de fibonacci (1, 1, 2, 3, 5, 8, ...)
- Ecrivez le noeud **Switch**(On, Off : bool) **returns** (s : bool) ;
 - s passe de faux à vrai si **On**
 - s passe de vrai à faux si **Off**
 - s est initialement faux mais peut être mis à vrai dès la première réaction
 - doit fonctionner correctement même si **On** et **Off** sont vrais en même temps
- Ecrivez le noeud **Count**(reset, x : bool) **returns** (c : int) ;
 - c est remis à zéro si **reset**
 - Sinon, c est incrémenté si x
 - c vaut 0 au départ

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1					
pre f	nil	1				
0 -> pre f	0	1				
f + (0 -> pre f)	1					
pre (f + (0 -> pre f))	nil	1				

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1	1				
pre f	nil	1	1			
0 -> pre f	0	1	1			
f + (0 -> pre f)	1	2				
pre (f + (0 -> pre f))	nil	1	2			

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1	1	2			
pre f	nil	1	1	2		
0 -> pre f	0	1	1	2		
f + (0 -> pre f)	1	2	3			
pre (f + (0 -> pre f))	nil	1	2	3		

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1	1	2	3		
pre f	nil	1	1	2	3	
0 -> pre f	0	1	1	2	3	
f + (0 -> pre f)	1	2	3	5		
pre (f + (0 -> pre f))	nil	1	2	3	5	

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1	1	2	3	5	
pre f	nil	1	1	2	3	5
0 -> pre f	0	1	1	2	3	5
f + (0 -> pre f)	1	2	3	5	8	
pre (f + (0 -> pre f))	nil	1	2	3	5	8

Lustre

- Fibonacci :

```
f = 1 -> pre (f + (0 -> pre f));
```

```
f = 1 -> 1 -> pre f + pre pre f;
```

t	0	1	2	3	4	5
1 -> pre (f + (0 -> pre f))	1	1	2	3	5	8
pre f	nil	1	1	2	3	5
0 -> pre f	0	1	1	2	3	5
f + (0 -> pre f)	1	2	3	5	8	13
pre (f + (0 -> pre f))	nil	1	2	3	5	8

- Switch :

```
node Switch(On, Off: bool) returns(s: bool);  
let  
  s = if(false -> pre s) then not Off else On;  
tel
```

- Count :

```
node Count(reset, x: bool) returns(c: int);  
let  
    c =      if reset then 0  
              else if x then (0 -> pre c) + 1  
              else (0 -> pre c);  
tel
```

```
node Count(reset, x: bool) returns(c: int);  
let  
    c =      0 -> if reset then 0  
              else if x then (pre c + 1)  
              else pre c;  
tel
```

- Modularité

- Une fois défini, un noeud peut être utilisé comme un opérateur classique
- Notation fonctionnelle
- Ex : que vaut A ?
A = Count(true -> (pre A = 3), true)
- En cas de sorties multiples :

```
node MinMaxAverage(x: int) returns (a: int);  
var min,max: int;  
let  
    a = Average(min,max);  
    min, max = MinMax(x);  
tel
```

Exercice : chronomètre

- 1 sortie entière : **time**
- 3 boutons d'entrée : **on_off**, **reset**, **freeze**
 - **on_off** allume et arrête le chronomètre
 - **reset** remet le chronomètre à zéro (s'il est arrêté)
 - **freeze** gèle l'affichage (si le chronomètre est allumé)
- Variables locales :
 - **running** : **bool**, une instance de **Switch**
 - **frozen** : **bool**, une instance de **Switch**
 - **cpt** : **int**, une instance de **Count**

Lustre

```
node Stopwatch(on_off, reset, freeze: bool)
returns (time: int);
var      running, frozen: bool;
         cpt: int;
let
    running = Switch(on_off, on_off);
    frozen = Switch(  freeze and running,
                     freeze or on_off);
    cpt = Count(reset and not running, running);
    time = if frozen then (0 -> pre time) else cpt;
tel
```

Lustre

- Horloges
 - Concilier contrôle et flot-de-données
 - Exprimer le fait que certaines parties du système s'activent moins souvent
 - Notion d'horloges flot-de-données (similaires aux horloges en circuit)
 - **Un noeud s'active selon l'horloge de ses entrées**
- **Echantillonnage** : opérateur **when**

X	4	1	-3	0	2	7	8
C	true	false	false	true	true	false	true
X when C	4			0	2		8

- Lorsque **C est faux**, **X when C n'existe pas**
- Attention : **nil** n'est pas la même chose que **non existant**

- **Sur-échantillonnage** : opérateur **current**
- Le sur-échantillonnage (projection) d'une horloge est parfois nécessaire

X	4	1	-3	0	2	7	8
C	true	false	false	true	true	false	true
Y = X when C	4			0	2		8
Z = current(Y)	4	4	4	0	2	2	8

Lustre

- **Horloge d'une instance de noeud = horloge de ses entrées**
- Echantillonnage des entrées $\Rightarrow$ ralentissement du noeud
- En particulier, échantillonner les entrées $\neq$ échantillonner les sorties

C	true	true	false	false	true	false	true
Count((r, true) when C)	1	2			3		4
Count(r, true) when C	1	2			5		7

- Exercice : chronomètre avec horloges
- Reprendre l'exemple du chronomètre en ajoutant 2 variables booléennes `cpt_ena` et `tim_ena`
- le type de `cpt` devient : `(cpt : int) when cpt_ena`

Lustre

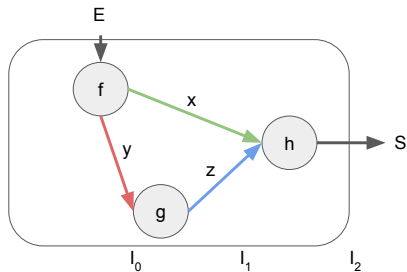
- Chronomètre avec horloges

```
node Stopwatch(on_off,reset,freeze: bool)
returns (time: int);
var      running, frozen: bool;
        cpt_ena, tim_ena : bool;
        (cpt: int) when cpt_ena;
let
    running = Switch(on_off, on_off);
    frozen = Switch(  freeze and running,
                    freeze or on_off);
    cpt_ena = true -> reset or running;
    cpt = Count((not running, true) when cpt_ena);
    tim_ena = true -> not frozen;
    time = current(current(cpt) when tim_ena);
tel
```

- Calcul d'horloges
 - Similaire au type checking
 - Les horloges sont nommées (même nom $\equiv$ unicité d'horloges)
 - L'horloge de chaque variable doit être définie (par défaut horloge **base**)
 - $\text{clk}(\text{exp when } C) = C \equiv \text{clk}(\text{exp}) = \text{clk}(C)$
 - $\text{clk}(\text{current exp}) \equiv \text{clk}(\text{clk}(\text{exp}))$
 - Pour tout autre opérateur **op** :
 $\text{clk}(e1 \text{ op } e2) = C \equiv \text{clk}(e1) = \text{clk}(e2) = C$

- Les horloges sont la bonne solution du point de vue sémantique (flot-de-données)
- Cependant les horloges sont potentiellement difficiles à utiliser
- Problème principal : initialisation
current(X when C) existe mais vaut nil jusqu'à ce que C soit vrai une première fois
- Solution (SCADE) : condition d'activation
 - pas un vrai opérateur (macro)
 - **X = CONDUCT(OP, clk, args, dft)** est équivalent à
X = if clk then current(OP(args when clk)) else (dft -> pre X)

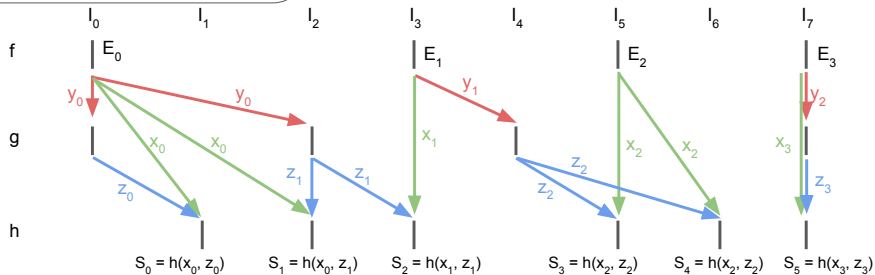
Lustre



$$C_f = [l_0, l_3, l_5, l_7]$$

$$C_g = [l_0, l_2, l_4, l_7]$$

$$C_h = [l_1, l_2, l_3, l_5, l_6, l_7]$$



- 1 noeud par sous-système
- Chaque noeud est cadencé sur l'horloge de ses entrées
- Exercice :
 - $f(E) = E, -E$; $g(y) = y, h(x, z) = x + z$;
 - $E0 = 0, E1 = 2, E2 = 6, E3 = 8$
 - Quelles sont les valeurs attendues des sorties ?
 - Comment spécifier Cf (ou Cg ou Ch) ?
 - Comment ralentir f (ou g ou h) ?
 - Ecrire le code correspondant en Lustre

Lustre

```
node f (E: int) returns (x,y : int);
let
    x = E;
    y = -E;
tel
node g (y: int) returns (z: int);
let
    z = y;
tel
node h(x,y: int) returns (S: int);
let
    S = x+y;
tel
```

Lustre

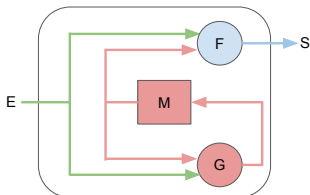
```
node test (E: int) returns (S: int);
var x, y, z, cnt: int;
cf, cg, ch, in: bool;
let
  cnt = 0 -> if pre cnt = 7 then 0 else pre cnt + 1;
  cf = (cnt = 0) or (cnt = 3) or (cnt = 5) or (cnt = 7);
  cg = (cnt = 0) or (cnt = 2) or (cnt = 4) or (cnt = 7);
  ch = (cnt = 1) or (cnt = 2) or (cnt = 3) or (cnt = 5) or (cnt = 6)
        or (cnt = 7);
  x,y = current(f(E when cf));
  z = current(g(y when cg));
  S = if in then 0 else current(h(x when ch,z when ch));
  in = true -> false;
tel
```

- Les langages synchrones sont réservés aux "noyaux" réactifs
- Pas très pratique pour les structures de données complexes
- On peut importer des types et fonctions abstraites depuis le langage cible (C)
- Existence de tableaux à taille fixée statiquement
- La récursion bornée est possible en Lustre v4
- Existence de modules et de template en Lustre v6

Éléments de compilation

- Schéma général d'une application réactive :

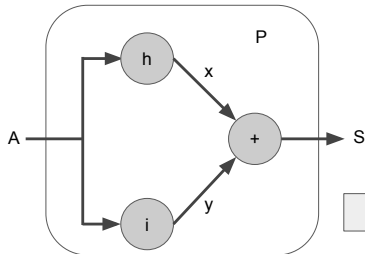
```
var I, O, M;  
M := m0; proc P_step()...;  
foreach step do:  
  read(I);  
  P_step();//O = F(M, I); M = G(M, I)  
  write(O);  
end foreach
```



- Déterminer la mémoire M et sa valeur initiale
- Ecrire le coeur de la réaction (P_step)
- Générer du code efficace autant que possible

Éléments de compilation

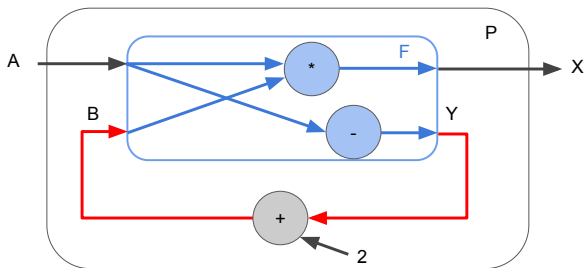
- Idée simple : 1 noeud = une fonction de réaction



```
var A, x, y, S;  
proc h_step()begin x := ... end  
proc i_step()begin y := ... end  
proc P_step()  
begin  
    h_step();  
    i_step();  
    S := x + y;  
end
```

Éléments de compilation

- Boucles de rétroaction ?



$X, Y := F(A, Y + 2)$

Boucle combinatoire !

Mais : $Y = -A$

=> inline :

$X, Y := F(A, 2 - A)$

$X := A * (2 - A)$

Plus de boucle

- Programme correct
- Compilation via `F_step()` ne fonctionnera pas

Éléments de compilation

- En Lustre : inlining statique
 - Complique/empêche la compilation modulaire
- En SCADE : pas de boucle sans retard synchrone
 - Rejette des programmes parallèles corrects
 - Permet la compilation modulaire
 - Choix industriel

Éléments de compilation

- Exemple : un compteur filtré
 - Compte les fronts montants de X
 - Remis à zéro avec un retard (R)

```
node CptF(X, reset : bool) returns (cpt: int);
var F, R: bool
let
  cpt = if R then 0
        else if F then pre cpt + 1
        else pre cpt;
  R = true -> pre reset;
  F = X -> (X and not pre X);
tel
```

Éléments de compilation

- Compilation vers une boucle (exécutif cyclique)
- Traduire les équations en assignations
 - Remplacer les opérateurs **pre** et $\rightarrow$ en constructions mémoires
 - Séquentialiser les assignations en respectant les dépendances (ordo statique)
 - Traduire les opérateurs classiques

Éléments de compilation

- Identifier la mémoire :
- Introduire une variable explicite pour chaque **pre**

```
pcpt = pre cpt;  
preset = pre reset;  
pX = pre X;
```

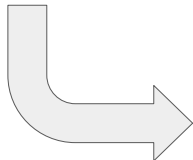
- Introduire une mémoire spéciale :

```
init = true -> false;  
  
x -> y devient if init then x else y;
```

Éléments de compilation

- Nouvelle version du programme :

```
node CptF(X, reset : bool) returns (cpt: int);  
var F, R: bool  
let  
    cpt = if R then 0  
          else if F then pre cpt + 1  
          else pre cpt;  
    R = true -> pre reset;  
    F = X -> (X and not pre X);  
tel
```



```
cpt = if R then 0  
      else if F then pcpt + 1  
      else pcpt;  
R = if init then true else preset;  
F = if init then X else (X and not pX);  
  
pcpt = pre cpt;  
preset = pre reset;  
pX = pre X;  
init = true -> false;
```

Éléments de compilation

- Séquentialisation :
 - Un ordre partiel doit exister entre les assignations (pas de boucle combinatoire) :
 $R < cpt$ et $F < cpt$.
 - Choisir un ordre total compatible avec cet ordre partiel (ordonnancement statique) :
R puis F puis cpt
- Mémoires :
 - Faites à la fin de la réaction, dans n'importe quel ordre

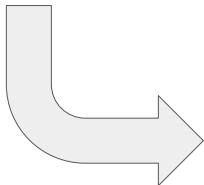
Éléments de compilation

- Traduction des opérateurs classiques (e.g **and** devient **&&** et **if-then-else** devient **... ?... :...**)
- Les **pre** sont remplacés par des mémoires
- Les **->** sont remplacés par **... ?... :...**
- Les entrées/sorties sont des variables globales

Éléments de compilation

```
cpt = if R then 0
      else if F then pcpt + 1
      else pcpt;
R = if init then true else preset;
F = if init then X else (X and not pX);
```

```
pcpt = pre cpt;
preset = pre reset;
pX = pre X;
init = true -> false;
```



```
int cpt; bool X, reset; // I/O global vars
int pcpt; bool pX, preset; // non initialisées
bool init = true; // seule init obligatoire
void CptFiltre_step() {
    bool R, F; // variables locales
    R = init ? true : preset;
    F = init ? X : (X && !pX);
    cpt = R ? 0 : (F ? pcpt + 1 : pcpt);
    pcpt = cpt;
    preset = reset;
    pX = X;
    init = false;
}
```

Éléments de compilation

- Optimisations :
 - Réintroduire des `if` et factoriser les conditions
 - Supprimer les variables locales inutiles

```
int cpt; bool X, reset; // I/O global vars
int pcpt; bool pX, preset; // non initialisées
bool init = true; // seule init obligatoire
void CptFiltre_step() {
    bool F; // variable locale
    if (init){
        cpt = 0;
        F = X;
        init = false;
    }else{
        F = X && !pX;
        cpt = preset ? 0 : F ? (pcpt + 1) : pcpt;
    }
    pcpt = cpt;
    preset = reset;
    pX = X;
}
```

Éléments de compilation

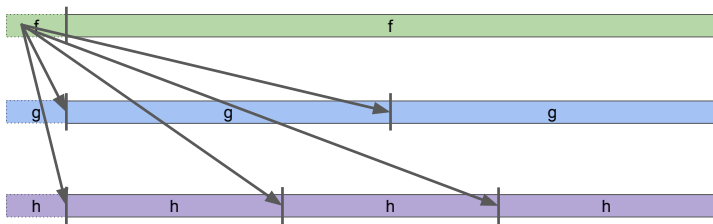
- Possibilité de compiler aussi sous forme d'automate
 - Plus rapide
 - Plus gros
- La boucle simple (exécutif cyclique) est la solution acceptable dans un contexte industriel
- Possibilité de réaliser de la vérification (plus simple sur automate)

Logical Execution Time (LET)

- Modèle "simplifié" de synchrone pour les systèmes industriels à tâches périodiques (en particulier automobile)
- Ex : Giotto
 - Moins expressif que les langages synchrones (pas de manipulation des horloges)
 - Synchronisations uniquement sur les débuts de périodes
 - Plus simple à utiliser
 - Spécification directe sous forme de tâches => sémantique de communication plaquée sur le système de tâches
- Ex : PsyC/Asterios
 - Possibilité d'insérer des points de synchronisation n'importe où dans le code
 - Version "contrôle" des synchros (vs flot-de-données)
 - Plus expressif, plus complexe à utiliser

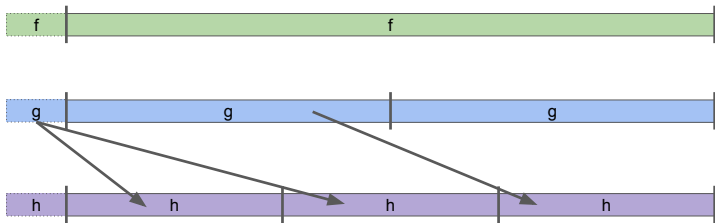
Logical Execution Time (LET)

Ex : Giotto



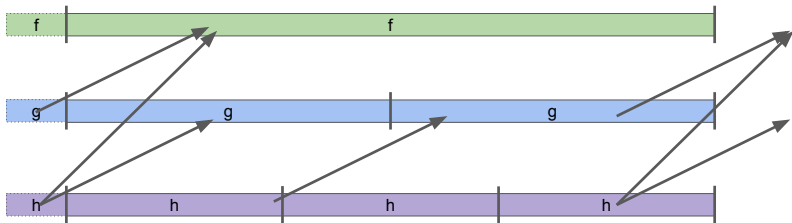
Logical Execution Time (LET)

Ex : Giotto



Logical Execution Time (LET)

Ex : Giotto



Logical Execution Time (LET)

Ex : Giotto

